

2IOE0 Intermediate Report: Stratego

Group 13, 2IOE0 Interactive Intelligent Systems 2020-Q2, TU/e*

ABSTRACT

In this paper the intermediate results and plans for further development of the game Stratego. This is done for the course ‘DBL Interactive Intelligent Systems’ in which a group of half a dozen students create a game using their knowledge of computer graphics and artificial intelligence, gained in the previous courses. A human player plays Stratego, on a virtual board, against an NPC. The NPC player is using two AI-techniques to play the game; Neural Network [6] for the positioning of the pieces on the board, and Monte Carlo Tree Search [8] for finding the best move. Each of the twelve pieces is uniquely modeled, so that a player can easily identify pieces on the board. The human player uses mouse to interact with pieces and to make moves, this is implemented using ray-casting [3]. Those are the two computer graphics techniques used in this implementation. Finally, the implementation and results are presented, as well as the reflection on those results followed by the potential improvements.

1 INTRODUCTION

While the game of Stratego is interesting in itself, this report and this course focus on the Artificial Intelligence (AI) and Computer Graphics (CG) components of the game. The goal of this project is to culminate the knowledge in AI and CG and present it in a form of a game with at least two visible CG components as well as two AI concepts which can be demonstrated to function. First in this report we present the game concept, the two AI and two CG techniques used to create the playable game, their implementation followed by results and conclusion. Finally we finish off by suggesting potential future improvements. This project is done for the course 2IOE0: Interactive Intelligent Systems at the Technical University of Eindhoven.

2 GAME DESIGN

Stratego is a turn-based strategy game where two players play against one another. Each player gets 24 pieces, with 12 different ranks, that they place in the first four rows of their side of the board. In this case a human player will face off against an NPC player. The game ends once a player has captured the opponent's Flag (a special piece, like king in chess), or when the opposing player can no longer move any units. This can occur when all remaining units consist of Bombs and the Flag, or when all other units are trapped within Bombs. At the start of the game both players create their formations in their designated part of the board, in such a manner that the opposing player does not know which piece is placed on a tile until that piece attacks or is attacked. The board has dimensions of 6x10 tiles where pieces can be positioned. Two pieces cannot be on the same tile at the same time. The Flag and Bomb are stationary pieces and do not move. Each player's pieces are recognizable by color. One player gets red pieces and the other blue pieces.

When both players finish creating their formations, the game can begin. Red has the first move. Each turn a player can move one of their pieces to either an adjacent empty tile, or an adjacent tile that is occupied by the opponent (except for the Scout, but more about

that later). The latter is called an attack.

An attack can have 3 outcomes:

- If the attacking unit has a higher rank than the defending unit, the attacking unit wins the encounter and captures the tile the defending unit was on. The defending unit is removed from the board.
- If the attacking unit has a lower rank than the defending unit, the defending unit wins the encounter and remains on the tile it was on before the encounter. The attacking unit is removed from the board.
- If the attacking unit has the same rank as the defending unit, the attacking unit and the defending unit both lose, and are both removed from the board.

Certain units are special pieces and cause exceptions to these outcomes of attacks:

- If the defending unit is a Bomb, the attacking unit loses the encounter. Unless the attacking unit is a Miner, in which case the attacking unit wins the encounter.
- If the attacking unit is a Spy and the defending unit is a Marshal, then the encounter is won by the Spy. This does not apply if the Marshal attacks the Spy.
- If the defending unit is a Flag, the game is won by the attacking player.

Other special rules when moving a unit:

- Units may only move back and forth 5 times, then another unit has to be used instead.
- Units may not “jump” over other units.
- Bombs and the Flag cannot be moved.
- Scouts can move any distance in a straight line (i.e. horizontal/vertical), as long as the spaces along the way are empty.

Initially, at setup each player has 24 units at disposal, distributed as follows:

Piece	Rank	Amount
Bomb	B	3
Marshal	10	1
General	9	1
Colonel	8	1
Major	7	2
Captain	6	2
Lieutenant	5	3
Sergeant	4	3
Miner	3	2
Scout	2	4
Spy	1	1
Flag	F	1

*members: Ahmad Alsarakbi, Mirre Bosma, Nick van den Brand, Edin Dzinic, Jeroen Oerlemans, Koen van Oijen

2.1 Changes from the original game

Due to the computational intensity of Monte Carlo Tree Search the size of the board proved to be an issue. A grid of 10x10 would simply not be viable, and MCTS could not compute a move in a reasonable time. Therefore thinking-time would have to be lowered and all-in-all the quality would go down. Thus, we have decided to shrink the board and to have only 6 columns. As a direct consequence the amount of pieces is lowered down to 24 per player, as 4 columns are removed. The last change we had made is to remove lakes completely and allow the middle of the board to be open. This was done to prevent having only 4 channels where the pieces can move in the beginning of the game.

3 CONCEPTUAL DESIGN

Layout of the game: When the player starts the game, the player sets up their board, or they select to be given a random starting board and once they are ready the game starts.

When the game has ended, a screen will be shown telling the user 'You lost!' or 'You won!'.

The board: The game is played on a 6 x 10 grid. The board is shown in figure 1. Blue pieces represent the human player's pieces as they are distinguished not only by color, but also by the fact that all the hats are shown, therefore all the pieces are known.

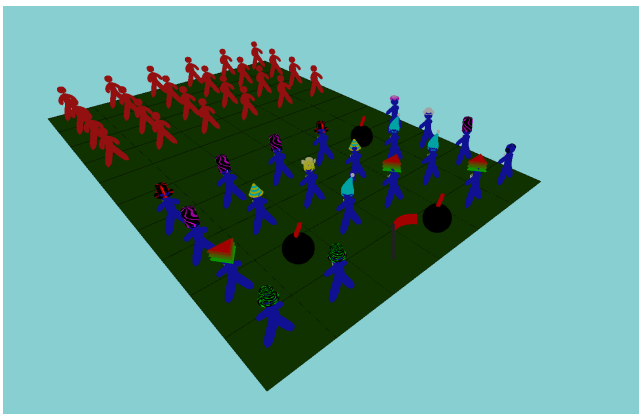


Figure 1: The board.

The characters: The characters, or pieces of different ranks such as Spy, Marshal or Miners, are represented by simple human like shapes, each having a different hat to distinguish themselves from others, as well as a roman numeral on their torso that indicates their strength. The shapes are red and blue for the NPC and human player respectively. In figure 2 we can see a known major with a number 7 on his chest.



Figure 2: Known Major

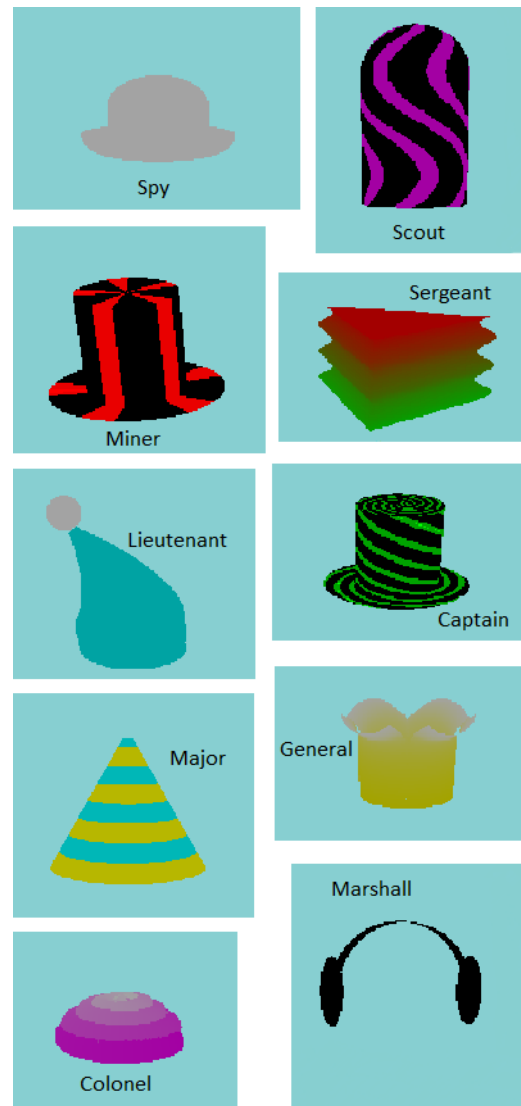


Figure 3: All the hats used in this game

A piece on the board can be in 2 states:

- Known: This way it is visible which character it is
- Unknown: This way it is not clear which character it is

The characters of the human player are always known (to the user), and as such are shown to the player. The characters of the opponent are unknown, in the beginning of the game, and become visible once they are attacked or attack a piece that belongs to the human player. When an attack takes place, the character of the opponent becomes known. The character that lost the fight will disappear from the board and screen entirely.

The character is represented by this simple human-like shape wearing a hat and a number on their body. The hats do not represent the traditional hats in the Stratego game. Instead, random geometric shapes and colors are used. This way, the distinction between the different units is clearer. Figure 3 shows all the different hats used to represent the units.

The bomb and flag are represented by bomb-like and flag-like shapes. Both are shown in figure 4

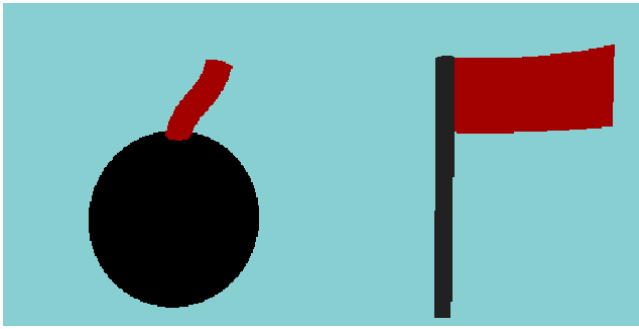


Figure 4: The bomb (left) and the flag (right)

Legend

In order to represent what each hat means, a roman numeral is rendered on the front of the torso of the piece. This is done for all the pieces except bomb and flag, both of which have distinct shapes.

4 ARTIFICIAL INTELLIGENCE

As the game of Stratego is played by two players playing against one another, in our case a player interacting with the game playing against an NPC. Therefore, all of the AI components implemented in this game will be focused on improving the gameplay of the NPC opponent. We divide the game into two phases: setup-phase and playing-phase. In the setup phase both human and the NPC player set up pieces on the board in a way in which they believe that will maximize their odds of winning the game. Position of both players' pieces is unbeknownst to the opponent. In the playing phase both players make moves, in turn, with pieces that they believe will give them an advantage. For us, humans, this can be almost straight forward in certain cases, but for a computer it can be almost impossible without using an AI to assist it. To enable our NPC player to act as a respectable opponent in the game of Stratego, a neural network will be used to set up the board of the NPC player, and Monte Carlo search tree will enable the NPC player to find an optimal moves in each state of the board.

4.1 Artificial Intelligence Technique I: Neural Network

In the game of Stratego, each player gets to place their forty pieces in the first four rows. Having twelve different types of pieces, it is necessary to have a way of creating an unpredictable setup whilst retaining equal or favorable odds of winning. For this to happen, a feed forward neural network (ref) is used.

Input/output: As an input the neural network has 24 nodes, each representing one of the tiles in the first four rows of the board, where the NPC places its pieces. The output of the board will be a single node, representing the score that the neural network gives to a selected setup, or in other words the input of that network. If this score surpasses a threshold, then that setup satisfies a certain criteria and is played. The criteria for the threshold can be determined before the game begins. If the human player decides to play against a hard AI, the threshold can be increased as opposed to playing against an easier AI.

The setup phase for the AI begins by randomly selecting a setup. Then, this setup is evaluated using the neural network. If the score is satisfying that setup is used in the game, if it does not reach the desired score a new random or pseudo-random setup is generated and evaluated again. This is done until a satisfactory setup is found. To increase the odds of finding the satisfactory setup a pseudo-random setup can be created. This is done by ensuring that the flag is placed in the last row and, and even further, that the bombs are in the vicinity of the flag. This would naturally increase the score of the setup.

Evaluation: To present that the setup AI is increasing the odds of the NPC-player, we will compare the simulated games against the completely random setups. If the win rate is increased for the AI using the setup neural network we can say that the AI is improved.

Training: To train the neural network we have to simulate games, and allow the network to learn. This AI goes hand-in-hand with the playing AI. Thus once the playing AI has been implemented, the neural network can use that AI to play the games, in the evaluation of the network. If a random setup plays the game and wins, we assume that the setup was good, at least for our playing AI, and the weights are adjusted accordingly. Analogously, if the game is lost, we assume that the setup was not compatible with the playing AI and the weights are adjusted. Note that both AIs cannot be trained at the same time, since playing AI needs to be trained first in for the setup AI to use it for the playing simulation rather than using random moves. This ensures that the two AIs work together to improve the strength of the NPC-player.

4.1.1 Implementation

To create our neural network we made use of the open source library Deeplearning4J (DL4J). The library gave us, aside from a large number of examples, the freedom to test with different kind of networks and lots of parameters. We started with an example template of DL4J and continued from there. The template was a convolutional neural network that was used as a MNIST classifier (source).

Input/Output: For the input, we will have an array of 24 (not 40 as was planned) doubles. Every unit got his own (normalized) value between 0 and 1. As can seen in the overview below:

- Flag - 1.00
- Bomb - 0.85
- Marshall - 0.765
- General - 0.68
- Colonel - 0.595
- Major - 0.51
- Captain - 0.425
- Lieutenant - 0.34
- Sergeant - 0.255
- Miner - 0.17
- Scout - 0.085
- Spy - 0

This will be put into our neural network and then be transformed to one single output node which will also be a number between 0 and 1. When the output is below 0.5 the setup will be classified as losing and above 0.5 will be classified as winning. The setups together with the actual classification were put inside a database, shuffled and 90% of the data would be used for training our data and 10% was used as test data. Every new trial the data was shuffled again.

Data collection: To create this data used for training, we made functions that create random setups, evaluate them and classify them into either winning or losing. Evaluation was done by letting a random setup play against 2000 good setups, 2000 bad setups and 2000 other random setups. How these good and bad setups came to be will be explained in a bit, but first more about the evaluation itself. For the neural network, it is important to have a lot of data. So we had

to find a way to get data as quickly as possible. Therefore, we had decided to let every simulation only consist of random moves.¹ The game would continue until the flag was found or, if the flag wasn't found, after 500 moves by evaluating the board in the following way:

$$\frac{(\text{Total strength player} - \text{Total strength opponent})}{\text{Total strength on board}}$$

Where the total strength is calculated by adding up the strength of every piece. When the value is above 0 the game was considered as won. Since a good setup is harder to beat than a bad setup, different type of opponents will give a different reward. A win against a good setup will give +0.8 and a loss will give a reward of -0.2. The same holds for a bad setup only vice versa, so -0.8 for a loss and +0.2 for a win. Lastly, a win from a random setup yields a reward of +0.5 and a loss -0.5. After 6000 matches, the average reward will be calculated and if the reward is higher than +0.2 the setup was considered to be winning and would be saved in a JSON file. To speed up the data generation even more, we mirrored the setup through along the y-axis after the evaluation, and gave them the same classification as the original and store them as well. Lastly, the creation of the good and the bad setups. For this we searched on the internet and saw that all good setups had their flag in the back row surrounded with bombs. For that reason, we let the setup generator first place the flag in last row and surrounded the three bombs around them. The rest of the pieces then would be assigned randomly over the rest of the available places. The bad setup would only place the flag in the first row (easy to capture) and let the rest of the free cells be assigned randomly.

In the end we gathered 6697 different setups of which 2076 (30%) were considered good setups.

Training: When we finally gathered our data it was time to start training our network. In this process we have tried two different networks: convolutional and fully connected. The biggest difference between the two, is that every node in a fully connected layer is connected to every node in the other layer and can be used for lots of application. Where the convolutional network has fewer and simpler connections, but it assumes the input to be images [5]. This is no problem, so for the convolutional network it will treat the setup as a image. After every epoch the network will spit out a evaluation matrix and a confusion matrix. An example is shown in the figure below.

In this case we have accuracy of 0.53 and a precision of 0.26,

```
=====Evaluation Metrics=====
# of classes: 2
Accuracy: 0,5317
Precision: 0,2636
Recall: 0,3277
F1 Score: 0,2922
Precision, recall & F1: reported for positive class (class 1 - "1") only

=====Confusion Matrix=====
  0  1
-----
261 162 | 0 = 0
119  58 | 1 = 1

Confusion matrix format: Actual (rowClass) predicted as (columnClass) N times
```

Figure 5: The output of the Neural network after an epoch

meaning the network classifies the setup 53% of the time, but when it classifies it as winning it will be right only 26% of the times. This

¹This is the same simulation that is used in the Monte Carlo Tree Search, so more about this in the second AI technique.

all can be verified with the confusion matrix below. Now we have the tools to evaluate our network we can start training it.

Starting with the convolutional neural network, the network consists of a number of hidden layers and an output layer. The l2 regularization parameter was set to 0.005, the weight initiation was set by the Xavier function, which set the weights randomly with some finite variance and mean equal to 0. The rest of the variable were changed in the process of trying to find the best network: the learning rate was always between 1E-3 - 1E-4. There were three types of layers used in the convectional network: Max pooling (reduces the size of the image, by filtering the image), dense layer (used in the fully connected network) and the ordinary convolution layer. Every layer consists of a activation, kernel size and the number of outgoing connections. Activation functions are equations that determine the output of a node. For the output layer the Sigmoid activation function was used to create an output between 0 and 1. Other activation function used were: Identity (a linear function), RELU, SELU and several others. The kernel size determines the size of the filter that goes over the image. Usually this is set to 3x3 or 5x5, but since our 'image' was not a square either we've tried rectangular filters as well. All these parameters were tested by trial and error. The result of the best network is discussed in the next section. However, when you want to see the results of more trials you can check out the logbook via the link: <https://drive.google.com/file/d/1ouSEkw2Sp3LanYE6q8d-IZXJ5HAtbEdX/view?usp=sharing>.

```
MultilayerConfiguration conf = new NeuralNetConfiguration.Builder()
    .seed(seed) //seedNetConfigurationBuilder
    .l2(0.0005) //remains the same
    .weightInit(WeightInit.XAVIER)
    .updater(new Adam(learningRate: 1e-5))
    .list() //NeuralNetConfigurationBuilder
    .layer(new ConvolutionLayer.Builder(kernelSize: 2, 4)
        //in and nOut specify depth. nin here is the nChannels and nOut is the number of filters to be applied
        .nIn(nChannels)
        .stride(1,1) //Determines the step size which the filter moves
        .nOut(48)
        .activation(Activation.RELU)
        .build())
    .layer(new SubsamplingLayer.Builder(PoolingType.MAX)
        .kernelSize(2,2) //
        .stride(2,2)
        .build()) //This takes groups of 4
    .layer(new ConvolutionLayer.Builder(kernelSize: 3, 3)
        .padding(1, 1)
        .stride(1,1)
        .nOut(24)
        .activation(Activation.IDENTITY)
        .build())
    .layer(new DenseLayer.Builder().activation(Activation.IDENTITY)
        .nOut(24).build())
    .layer(new OutputLayer.Builder(LossFunctions.LossFunction.NSGD)
        .nOut(outputNum)
        .activation(Activation.SIGMOID)
        .build())
    .setInputType(InputType.convolutionalFlat(height: 4, width: 6, depth: 1)) //See note below
    .build();
```

Figure 6: A convolutional neural network containing 5 layers with a Dense layer, a max pooling layer and 3 convolutional layers

Finally, we have created a fully connected network. The big advantage for this is that the network does not need to know the structure of the input and is therefore very easy to implement. For this type of network, we only have one type of layer, the dense layer. However, the number of layers, activation function and number of hidden nodes were all modifiable. In our case, we often used the same activation function as for the convolutional network and the number of hidden nodes gave the best results when it set around 32. So this was used most of the tries.

4.1.2 Results

In order to evaluate the Neural Network, a custom evaluator was built. The main idea behind the evaluator was the following: a random setup was created, then this setup was evaluated in the same way as the setups that were created for the neural network, and finally this evaluation is compared to the evaluation given by the Neural Network. If the neural network and MCTS-random evaluator agreed on the evaluation, that is considered a correct output. Naturally

```

int numHiddenNodes = 32;
MultiLayerConfiguration conf = new NeuralNetConfiguration.Builder()
    .seed(seed) NeuralNetConfiguration.Builder
    .weightInit(WeightInit.XAVIER)
    .updater(new Adam(learningRate: 1e-3))
    .list() NeuralNetConfiguration.ListBuilder
    .layer(new DenseLayer.Builder().nIn(nChannels).nOut(numHiddenNodes)
        .activation(Activation.IDENTITY).build())
    .layer(new DenseLayer.Builder().nIn(numHiddenNodes).nOut(numHiddenNodes)
        .activation(Activation.RELU).build())
    .layer(new DenseLayer.Builder().nIn(numHiddenNodes).nOut(numHiddenNodes)
        .activation(Activation.SELU).build())
    .layer(new OutputLayer.Builder(LossFunctions.LossFunction.MSE)
        .activation(Activation.SIGMOID)
        .nIn(numHiddenNodes).nOut(outputNum).build())
    .build();

```

Figure 7: A fully connected network containing 4 layers with 3 different activation functions

these results have to be taken with some doubt as simulating games with MCTS instead of random moves would give richer results, but the computational time would simply be too great to get any large amount of results. Using this we have a 200-setup evaluation, which can be found here <https://drive.google.com/file/d/1KFeIrSuNoPqrWYSlbVVR3XpfQIQ4sVs0/view?usp=sharing>. Explanation of the data: the first column represents the Neural network estimator and the second column represents the MCTS-random estimator. However, we are only interested in one number and that is the actual accuracy of the model. In the run where we evaluated 200 setups, we ended up with an accuracy of 45%. This number varied during multiple runs (only one run was unfortunately saved), but the accuracy varied between 42-50%. This means that the neural network would produce a winning setup almost half of the times. This is a slight upgrade compared to a randomly generated setup, in which case we get a good setup only 30% of the time, as shown when generating data for the neural network. Further improvements will be discussed in Section 6.1.1.

4.1.3 Conclusion

We have showed how we used and implemented the Neural Network in the setup phase. We showed implementation, data collection and training of the network. After testing we found out that we managed an improvement from 30% to 45% over the random board setups. All-in-all the improvement was not the greatest and leaves a lot to be desired, but it is an improvement over the random setup nevertheless. We shall discuss potential improvements later on in the paper.

4.2 Artificial Intelligence Technique II: MCTS

The second AI technique will be used once the setup is done and the units need to be moved. For this part of the game Monte Carlo tree search (MCTS) (ref) algorithm is used. This algorithm is often used in board games and other games with incomplete information. Since Stratego is a combination of those two types of games MCTS seems to be a suitable option. The algorithm will be used to solve the game tree by looking for the most promising nodes and exploring those moves deeper than less propitious nodes. In this game tree the root is the current game state and the leafs are either the state of a finished game or a parent node of a (potential) child where no simulation has yet been initiated. Each round of the MCTS consist of 4 steps:

1. Selection
2. Expansion
3. Simulation
4. Backpropagation

In the first step, the selection step, a leaf will be chosen from the tree. This choice will be based upon the balance between the most promising and the least explored move. This balance is important. If we only pick the most promising move, the moves that initially do not look very good, but are good on the long run would be completely neglected. Then in the expansion step, a child node will be created from the parent node that just has been selected in the previous. From this expansion, a random roll-out/play-out will be done in the simulation step. In a roll-out the game will be finished by selecting random moves. Then during the backpropagation, the weight of each node will be changed depending on the result of the roll-out (a win or a loss). Since the opponent players are also unknown, the role of each unit will also be assigned randomly. In case the roll-out turns out in a draw, for example after 50 consecutive moves without a piece being attacked, the algorithm will look how many and which pieces are left and based upon that will 'throw' a weighted dice to pick a winner.

MCTS has a few big advantages. First of all, it can be stopped at any random moment and will return with the most promising move yet found. Next to that, we do not have to think of an evaluation function, since the roll-out will give either win or loss. Lastly, it will remember the branches that the algorithm already visited and this will be taken in consideration for the next move and will save some computation time. This technique has a few drawbacks. Firstly, because there are a lot of possible moves and unknown characters, especially at the beginning, the roll-outs will need to make a large amount of guesses or assumptions. This means the AI does not function very well at the beginning and will be just as good as playing against an opponent who plays random moves. However, when the game continues there will be more knowledge about the game state and the evaluations will become more accurate. A second drawback of the algorithm is that there are not a lot of examples of MCTS being used for Stratego, so it is difficult to predict how well this algorithm will perform. Right now the best AI's for Stratego do not score very high against human players. [4] However, we want our AI to, at least be able to beat a random player (a player just makes random moves) more than 60% of the time. (ref) Another challenge is to give the algorithm the right time to find the next move. If it takes too long, the game will not be enjoyable to play, but if it spends too little time looking for the move, it will not necessarily find the best possible move. This means that a good balance is needed and this can only be found by testing out different parameters.

4.2.1 Implementation

This section will be split into 4 parts, one part for every step mentioned in the section before.

Selection: To start with the selection phase. Here a leaf will be picked from the tree and from there will try to expand the tree. To keep the balance between exploration and exploitation, the formula called UCT (Upper Confidence Bound 1 applied to trees) has been used. [7]

$$\frac{w_i}{n_i} + c \sqrt{\frac{\ln N_i}{n_i}}$$

- w_i : Wins after the i^{th} node
- n_i : Number of visits after the i^{th} node
- N_i : Total number of visits of the parent node after the i^{th} node
- c : A constant. In our case set to 0.5

The wins are based upon the simulations that are done in the third step, which are propagated back in the fourth step of a previous run through. After looking at all children, the node with the highest value will be expanded in the next step.

Expansion: When a leaf has been chosen by the function above, new children will be created from this node. The children are all the possible moves that can be made from the current board state. These moves are found by a function that looks for every movable unit and looks if it can move in a single direction. Then for every single child will be doing a roll out.

Simulation: A roll out or simulation is consists of maximum of 500 moves, just like the gathering data part in the previous chapter, and will consist only of random moves. The pseudo code of the simulation looks as followed:

Algorithm 1 Simulation

Input: Board b, Player p

Output: reward between -1 or 1

while game is not finished **do**

 List moves = GetLegalMoves(b, p)

 move = moves(Random)

 b = MakeMove(move)

 player = (player+1) % 2

return Reward(b)

Where the reward will be a double between -1 (game lost) and 1 (game won), which will be calculated in the reward function. This function will add up every unique value of every piece, which can be found on the first page of this report (the bomb has a value of 8). Since a lot of pieces are unknown for the player, the tree will assign the not discovered pieces to the possible cells. When a piece has moved it can not be a bomb or a flag anymore (because they can't move) and will be taking into consideration as well. This allocation of unknown pieces will be done after every (actual) new turn. furthermore, to make the reward function more sophisticated, the value of a miner and the spy will be increased, when there are still bombs or a marshal (respectively) in the opposite team. Finally, when the flag is not found in either team the function will return -1 or 1 instantly, but if the flags are still there the value of your team will be subtracted by the value of the opponent's team divided by the total value. Exactly the same as for the data collection for the neural network.

Backpropagation: When the simulation is done the node and its parental nodes need to be update. This will be done by the backpropagation function, which looks as followed:

Algorithm 2 Backpropagation

Input: Node n, Reward r

Output:

while $n_{parent} \neq null$ **do**

if $n_{visits} == 0.1$ **then**

$n_{visits} = 1$

else

$n_{visits} += 1$

$n_{quality} += reward$

$n = n_{parent}$

if $n_{visits} == 0.1$ **then**

$n_{visits} = 1$

else

$n_{visits} += 1$

$n_{quality} += reward$

The reason n_{visits} is set to 0.1 is that some unvisited are being used to calculated the UTC. However, since dividing by 0.1 is not

possible an arbitrary small number has been chosen.

When all 4 steps has been passed through and there is still time left (usually the tree has a thinking time of 0.5 sec per move), the tree will repeat the process all over again for the next turn. When the time is up it will the UTC evaluation for the root (current board position) and will pick the best child and do that move. So our complete algorithm looks as followed:

Algorithm 3 Monte Carlo Tree Search

Input: Node root, Node current, Int player, starttime, endtime

if Game is finished **then**

return evaluateUTC(root)

Children = current.getLegalMoves;

for child : Children **do**

 result = simulatechild.board, player

 backpropagate(child, result)

 Update starttime

if starttime > endtime **then**

return evaluateUTC(root)

player = (player + 1) % 2

return MonteCarloTreeSearch(Root, EvaluateUTC(current), player, starttime, endtime)

4.2.2 Results

When two NPC players, both of whom make random moves, play against each other the win rate of either player is about 50%, as expected. Our main goal was to increase this win rate for our NPC player using MCTS. To show that this is achieved, we built once again a custom evaluator. This evaluator creates one random setup, and assigns it to both players, this way both players are using the same setup and the only variable that comes to play is the moves that the players are making. One of the players is using random moves, and the other is using MCTS. MCTS in this case MCTS was using a high thinking time of 5000ms, and a move limit of 500 moves. This thinking time reflects a thinking time that a NPC player might have playing against a human player. We ran this test 200 times, using 200 setups. The results can be found here <https://drive.google.com/file/d/1J0GJXAXu7SS1c8U1L6EZs96s560zsvP-/view?usp=sharing>. First, each board is displayed, followed by the total reward (calculated by the reward function described in the previous section) and finally the winner. Since the game ends after 500 moves, the player with more material on the board wins, therefore if the reward is negative the random player wins, and if the reward is positive MCTS player wins. There is a possibility of a complete tie, in case the reward is equal to 0. After 200 runs, the total win rate is 63,5%. This is again an improvement compared to random moves. Potential further improvements will be discussed in Section 6.1.2

4.2.3 Conclusion

In this section we discussed how we implemented the Monte Carlo Tree Search for the adjusted game of Stratego. We have seen an improvement by getting a win rate of 63%. However, MCTS being a new technique which we had to familiarize ourselves with and implement and test for the first time, we believe that the improvements could have been greater. Having few similar problems to compare to implementation proved to be difficult for us, and most of the time was spent search for, fixing issues and adjusting small things.

5 COMPUTER GRAPHICS

To add a different feel to the game than the standard real life game we are going to use computer graphics techniques to implement complex models of the pieces, as well as adding flying camera point of view (POV) that can be fully controlled via the mouse, so as to give the player more freedom when playing the game.

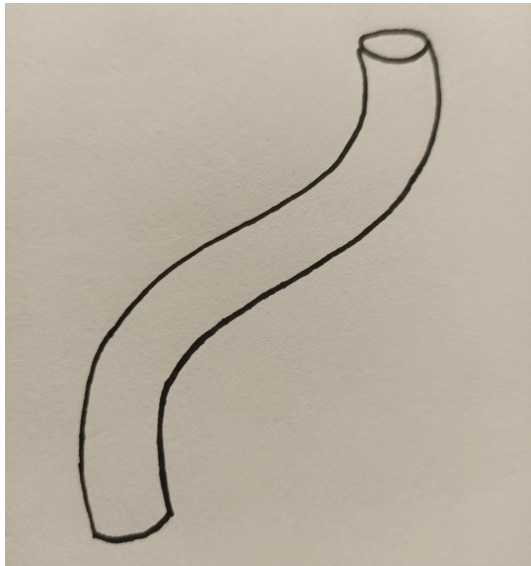


Figure 8: Example extruded figure

5.1 Computer Graphics Technique I: Modeling

As discussed in game design, each type of character is uniquely identified, that's why we are going to build a unique model for each piece.

To build complex models that reflects each character, modelling techniques and algorithms (without using 3d party software) are used. Two techniques that are used are:

- Extrusion: using 2d polygons and shapes as a base for 3D models, by extruding, rotating and sweeping these 2d shapes we can create complex 3D models with relative ease.
- Bézier curves and surfaces: using the Bézier curves and surfaces to model more complex shapes that have non-trivial curves. Some models might not be realisable through the extrusion of basic shapes and polygons that's why we are going to use Bézier curves to realise these shapes.

We will construct a general approach to make each piece. Once we have this basic approach, we will use it to make all the individual pieces.

5.1.1 Implementation

Using extrusion, all the figures that you can possibly dream of can be made. Extrusion is done using a base form and a curve along which this base is 'extruded'. Figure 8 shows a circle that is extruded along a curve. We will use this example as guide to explain how the extrusion is implemented.

In the list below, the parameters are given that are needed to implement extrusion. These parameters will later be explained more explicitly.

- points on a shape
- tangents of the points on the shape
- points on a spline
- size for each point on the shape
- rotation for each point on the shape

- a matrix with the color values for each point on the spline and shape

Let us use the example given in figure 8. The shape that we want to extrude, in the example this is the circle, is called the spline. This is always a 2D curve. The curve along which we want to extrude is called the shape. The shape is a 3D curve. The spline is represented in figure 9 and the shape in figure 10.

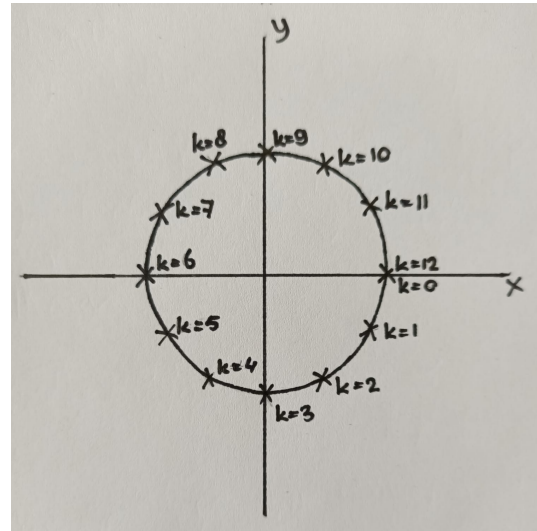


Figure 9: Spline for the extrusion

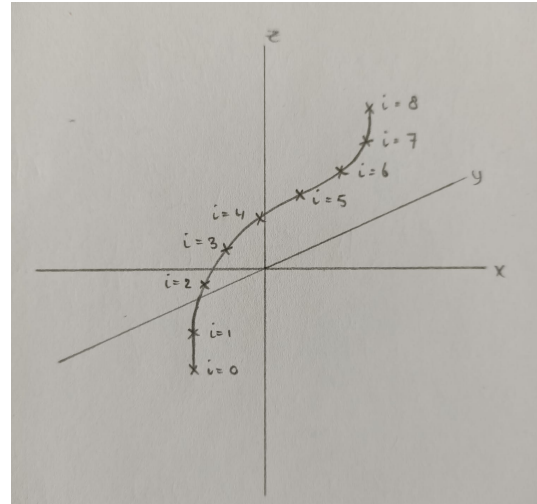


Figure 10: Shape for extrusion

As is visible in the pictures, on both the spline and the shape, points are defined. The points on these curves can be chosen manually, or using Bézier curves. The larger the number of points on each curve, the smoother the surface will look. For each point i on the shape we are going to make sure the tangent at point i is perpendicular to the surface defined by the 2D curve, the spline. After this is done, the transformed points on the spline should look like in figure 11. The pink line represents the shape and the green one the spline.

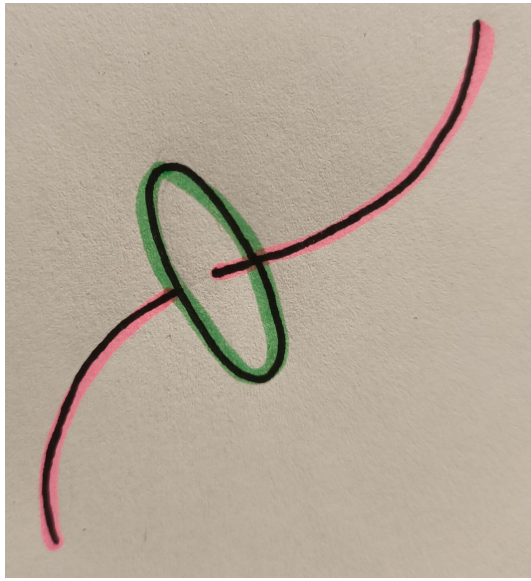


Figure 11: Caption

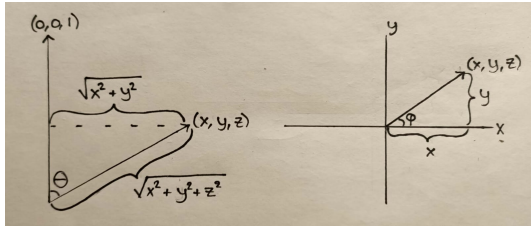


Figure 12: Calculation of the degrees of rotation

This transformation is accomplished by using transformation matrices: rotation around the y-axis, rotation around the z-axis and translation. Given the tangent $t = (x, y, z)$, the degrees for rotation are calculated as follows:

$$\text{rotation } y = -\text{sgn}(x) * \text{sgn}(z) * \arcsin\left(\frac{\sqrt{x^2 + y^2}}{\sqrt{x^2 + y^2 + z^2}}\right)$$

$$\text{rotation } z = -\text{sgn}(x) * \text{sgn}(y) * \arctan\left(\frac{|y|}{|x|}\right)$$

if $(\text{sgn}(x) * \text{sgn}(z) = -1)$, then

$$\text{rotation } z = \text{rotation } z + \pi$$

We can visualize this as in figure 12. (x, y, z) are the coordinates of the direction of the tangent. On the left side we see the calculation for the degree of the y rotation. On the right side the calculation for the rotation around the z-axis.

First, the y-axis rotation is applied. Next, the z-axis rotation is applied. And finally, the translation is applied.

This rotation is calculated for two points at a time: i and $i + 1$, where $0 \leq i < \text{number of points on the shape}$. Then, for every k and $k + 1$ on the two splines (now transformed), a rectangle is drawn. The result of this is seen in figure 13. Again, the pink line represents the shape and the green line the spline.

All this rectangles together (at every i and every k on the shape and spline respectively), form the shape drawn in figure 8.

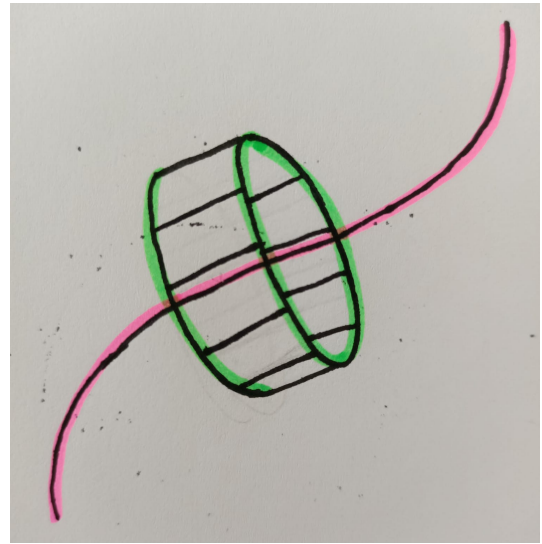


Figure 13: The spline after applying transformation

Now, also special actions can be applied. The shape of the size at each point on the shape can vary. The spline can be 'pre-rotated' around the z-axis at every point on the shape. And, every rectangle can be assigned a different color. We will show these applications by using an example. Figure 14 shows us the hat that defines one of the models.

The following model is made using extrusion.



Figure 14: Model made using extrusion

This hat is built from 1 shape and spline. Hats can also be made using multiple splines and shapes. Now, first, the points for the spline (in this case, again a circle) and the shape (in this case a straight line), are defined. Next, the color, size and pre-rotation are defined.

The color is defined for every single rectangle, represented by the

points (i,k) on the shape and spline and for every RGB value. The color per rectangle is saved in a matrix. Define n = the total number of points on the spline. Then, for this hat, for the R and B value, the color $c[i][k]$ is calculated by $c[i][k] = \lfloor \sin(\frac{k*12*\pi}{n}) * 2 \rfloor$. For the G value, the color $c[i][k] = 0$.

Rotation and size are defined only for each point on the shape, and not for the points on the spline. Let m be the total number of points on the shape. For this specific hat, the rotation R is calculated by $R = \sin(\frac{6*\pi}{m})$. For every i , this value is saved in an ArrayList.

Size works the same as rotation. For each i , the size is calculated and saved in an ArrayList. For this hat, the size S is calculated as follows: $S = 0.48 * \sqrt{1 - (1 - \min(\frac{4*i}{m}, 1))^2}$. Then, using transformation matrices, these values are applied to every point on the shape. These two transformation matrices are applied before the rotation around the y-axis, rotation around the z-axis and the translation matrix.

Animations can also be made using the extrusion technique. This can be done using a time parameter. This time parameter can be used inside the definition of points or inside the functions to calculate size, color or rotation.

5.1.2 Results

The results for this technique can be seen in figure 3 and figure 4 in chapter 3 about the conceptual design.

5.2 Computer Graphics Technique II: Ray-Casting

Another CG technique that needs to be used in order to make a working game that is also easy and fun to play, is Mouse Control implementation, and for this we need to project the mouse movement and interactions into the 3D world coordinate system (Ray-Casting). Mouse control implementations that will be used are:

- Mouse to world projection: project the mouse position in the 2D screen onto the 3D game world, and move the pieces around with the mouse.
- Screen edge dragging: recognize the screen edge and dragging the scene when the mouse is at the screen edge and is still moving. This is to make it easy for the player to move around the board if the game is zoomed in.
- Scene rotation: rotating the scene according to mouse movement when a special criteria is met (e.g. right or middle mouse button is pressed), to give the player the freedom to look at the game from different angles
- Zooming: zoom in and out to make it easy to change the scope of the player view

5.2.1 Implementation

5.2.1.1 Mouse projection into world coordinate: To be able to move the pieces on the board using the mouse 2 steps are needed. The first step is to project the mouse 2D screen coordinates into the 3D world and then create a ray from the camera position to the projected position. For this step we follow the following procedure: [2]

- The first thing to do is reading the mouse click event coordinates from the mouse event listener and share these with a global variable.
- Then these variables are normalized from the ranges $x = [0, width]$, $y = [0, height]$ to $x, y = [-1, 1]$ so that it's easier to work with different view port sizes.
 $x = (2.0 * mouse_x) / width - 1.0$
 $y = 1.0 - (2.0 * mouse_y) / height$

this is also because we don't really care about the exact point in the world coordinate since the projection is used to cast a ray.

- Then the homogeneous clip vector is create using the normalized x and y :
 $vec4ray_clip = vec4(x, y, -1.0, 1.0)$
 -1 was used for the z coordinate here to indicate looking forward along the z axis, but in reality it doesn't effect our technique if we use 1 instead since we just need two points along the ray to construct it, because of that we can also skip reversing the perspective division since depth holds no real effect on what is done next.
- since usually a vector is multiplied by the projection matrix to transform it from eye coordinates to clip coordinates, to go in the opposite direction we can multiply by the inverse of the projection matrix to get our mouse homogeneous clip coordinates into eye coordinates. $vec4ray_eye = inverse(projection_matrix) * ray_clip$ and since this is just a direction vector to ray cast from camera position and not actually a point the z and w are changed to -1 and 0 respectively to indicate that this vector is a direction vector and that it is facing forwards.
- the last step is to transform the vector from eye coordinates into world coordinates this is also just multiplying by the inverse of the view matrix:
 $vec3ray_world = (inverse(view_matrix) * ray_eye).xyz$
and then by normalizing this vector we get a directional unit vector inside world coordinates that points from the camera position to the mouse click position in the view port:
 $ray_world = normalise(ray_world)$

using ray_world and the camera position we can construct a vector in the world space that we can extend forwards or backwards and intersect with the different hit-boxes. we can also construct a line along this vector and use this in the calculations of intersection points with planes. since our game is a simple board game most units are moved inside board cells which makes box shaped hit-boxes easier and less computational extensive to calculate. The intersection technique is explained in the next section.

Algorithm Ray-Casting(m, p, S)

```

1.   $r \leftarrow$  direction vector between mouse point  $m$  and projected point  $p$ 
2.   $D \leftarrow$  Empty array of shapes
3.  for all shapes  $d$  in  $S$ :
4.      Get point  $Q$  and  $R$  from two opposite corners of  $d$ 
5.      for all surfaces  $s$  in  $d$ :
6.           $c \leftarrow$  intersection point between  $s$  and  $r$ 
7.          if  $c$  lies in  $s$ :
8.              add  $d$  to  $D$ 
9.   $L \leftarrow$  Empty array of distances
10. for all shapes  $d$  in  $D$ :
11.      $l \leftarrow$  distance between  $m$  and  $d$ 
12.     add  $l$  to  $L$ 
13. return shape  $d$  with minimum value in  $L$ 

```

Figure 15: Ray Casting Psuedocode

5.2.1.2 Ray Casting: As seen in figure 15, a direction vector is computed between the mouse click world coordinates and the camera position. Since all our shapes are computed in box-form, a point Q and R can easily be computed in the following way:

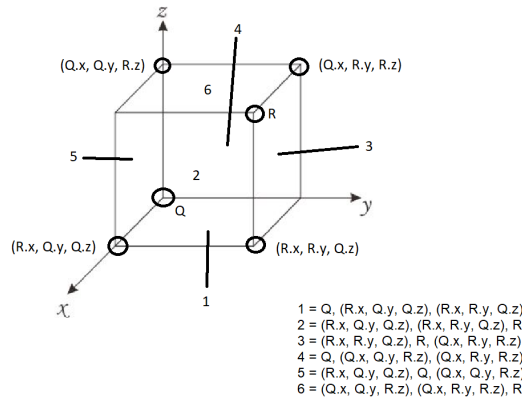


Figure 16: Ray Casting Square And Surface Computation

Figure 16 also shows the 4 points derive from Q and R to create the place vectors of the 6 surfaces. This is done by computing the normal of the 3 points on the surfaces. Lets say (a, b, c) . Then the plane equation becomes $a * (x - x_0) - b * (y - y_0) + c * (z - z_0) = 0$ with (x_0, y_0, z_0) as one of the 3 points on the surface. With this equation, if we have the direction vector between mouse point m and projected point p , we get the direction vector equation $(m_x, m_y, m_z) + t * (p_x - m_x, p_y - m_y, p_z - m_z)$ [1] substituting the line equation in the plane equation gives us the intersection point. If that point lies in the original surface, then Ray-Casting returns that shape as one that intersects. As final computation, it computes all the distances between the returned shapes and the mouse point. It takes point Q and R again and computes the center point between these points, lets say point S. The distance is then calculated by the following formula: $\sqrt{(m_x - S_x)^2 + (m_y - S_y)^2 + (m_z - S_z)^2}$. The shortest distance is returned and that shape is the one highlighted by Ray-Casting.

Note: only hit boxes around the cells were implemented.

5.2.1.3 Screen Dragging: To implement this we first need to recognize the edge of the game window, however that proved to be challenging without locking the mouse to the inside of the game window.

This however requires the use of some very heavy computations on the cpu part, to avoid this unnecessary overhead this the game window was set to full screen.

And to avoid locking the player in a full screen game with no way out the Esc button was set to exit the game.

When the mouse reaches one of the edges it's x position is equal to either 0 or $screen_width$ and it's y position is equal to either 0 or $screen_height$, to differentiate between movement along the x and movement along the y we introduce two global variables $cameraMovementSpeedX$ and $cameraMovementSpeedY$ that represent the speed of dragging, these variables are 0 when the mouse is not at the edges.

When the mouse reaches the right edge of the screen and it's x coordinate is equal to $screen_width$, $cameraMovementSpeedX$ is set to $-speed$ and when the mouse reaches the left edge of the screen and it's x coordinate is equal to 0, $cameraMovementSpeedX$ is set to $speed$.

When the mouse reaches the lower edge of the screen and it's y coordinate is equal to $screen_height$, $cameraMovementSpeedY$ is set to $-speed$ and when the mouse reaches left edge of the screen and it's y coordinate is equal to 0, $cameraMovementSpeedY$ is set to $speed$.

We then calculate the vector *towards* between the camera *center* and camera *position* to make sure the the movement is parallel to the scene alignment other wise the movement would be skewed . Then a x movement vector *towardsX* is calculated from the *towards* vector by multiplying it by the *cameraMovementSpeedX* and the same is done to calculate a y movement vector *towardsY* by multiplying the *towards* by the *cameraMovementSpeedY*.

But this would result in both vectors *towardsX* and *towardsY* that are equal to each others and both would simulate a vetical movement.

To simulate a horizontal movement we need to find a vector perpendicular to these vectors and since *towardsX* is the vecotr that should simulate the horizontal movement we need to adjust it.

This can be done by swaping it's x and y coordinates and then negating it's x as such:

$towardsX = newVec3(-towardsX.y, towardsX.x, 0)$

because only vetical and horizontal movements are needed for dragging we set the z coordinate of both *towardsX* and *towardsY* to zero.

Then by adding these 2 vectors to the *position* and *center* vectors of the camera the desired effect of dragging the scene is produced.

5.2.1.4 Screen rotation and zooming: These techniques were implemented for the course computer graphics and the implementation inside that course were reused in this project.

5.2.2 Results

The results of this technique is a full mouse control for our game with the four mentioned components that can be tested using the game. We also created a tool to draw a line that resembles the ray casted by our ray casting algorithm to show where it's pointing at as can be seen in figure 17.

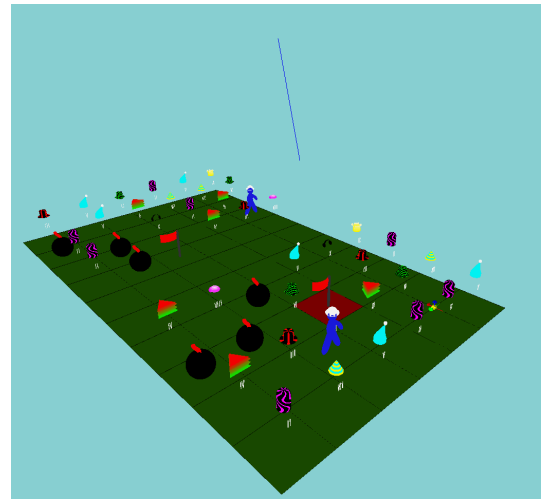


Figure 17: Ray drawn when clicking the red square

6 RESULTS

At first the game was altered from the original game of stratego. This was done due to the limitations of our AI.

6.1 Potential Improvements

In this section we discuss the potential improvements for the techniques used in this project.

6.1.1 Neural Network

Our neural network has not worked as we hoped and this could have multiple reasons and so there are multiple improvements that could be made. The first one being the an improved data set. Right now, our dataset only consists of only 30% of winning setups. Therefore, after several epochs the network almost always started to predict everything as losing, since this would give an accuracy of .7. So adding more winning setups could have been one the things we could have tried. Secondly, we could have made the data not binary (winning or losing), but rather given it a value between 0 and 1. Because if we take a look at the results we saw that our network had a lot of difficulties predicting the edge cases, where the value was around the division point. Maybe this would give the network more to work with and could give some better predictions. Lastly, our data could also not make sense at all, and our classification could be incorrect in the first place. This has never been done before and we had to make quite a few assumptions. Maybe, it would have been better to not let the player make random moves, but rather use the MCTS to let them play out the game.

6.1.2 MCTS

The first idea for improving the MCTS is to assign weights to the pieces dynamically. This is best reflected in example where one player has a miner and the other has a bombs. Since miners can remove bombs they are worth more if there are opponent bombs present on the board, and they are almost useless without bombs, they would only beat out the spy. Analogously, this can be applied to spies and marshalls, as spy beats out marshall if he attacks the marshall. This would make MCTS value its spies and miners more and would not aimlessly sacrifice the miners and spies and lose the ability to remove bombs and marshall from the board.

The second idea for the improvement of MCTS is to shuffle the board, when simulating moves, only when a piece which has not previously moved moves, or when a piece which is unknown becomes known. This would prevent too much randomness interfering with the decision of the MCTS. This should make MCTS more decisive, as it would believe the flag is in a fixed position for a longer time than it is now, as now each (simulated) move it is potentially moving the flag around the board.

6.1.3 Mouse controls

Implementing hit-boxes for the model parts would make clicking much easier and more intuitive. Also the dragging is rather slow when the screen is in top down angle, this can be improved by making movement speed of dragging scale with the angle.

7 CONCLUSION

Looking back at the result, we are satisfied with the end result. The four chosen techniques were all implemented in time and the game looks quite good for an 8 week project. That said, the results for the AI techniques were disappointing, yet not surprising. Until today there still does not exist a stratego AI which is able to beat humans, so even a small improvement in win rate can be seen as a success. The bigger improvement could have been achieved for the Neural Network, and for the MCTS it was difficult to predict the outcome, and having a new technique to implement, we consider it a lesson learned and a success. For the CG techniques mouse control was implemented successfully, however the lack of smaller hit-boxes make clicking from certain angles a bit of a challenge, and also we had planed on implementing those it was a bigger challenge for us than anticipated because extruded shapes range limits are harder to calculate than normal shapes, as for extrusion it was an unfamiliar technique especially with bezier curves yet it was implemented to a high degree of success, the only problem is that performance drops fast when increasing the resolution of the models, which forced us to lower the number of points per bezier curves.

REFERENCES

- [1] P. Dawkins. Calculus iii, Nov. 2018.
- [2] A. Gerdelan. Mouse picking with ray casting, 2016.
- [3] P. Lemons. Ray casting algorithm. 2017. doi: posts/ray-casting-algorithm
- [4] I. S. Maarten Schadd. The 2nd stratego computer world championship. https://www.researchgate.net/profile/Maarten_Schadd/publication/238750010_The_2_nd_Stratego_Computer_World_Championship/links/54d47f0a0cf2970e4e634928/The-2-nd-Stratego-Computer-World-Championship.pdf, 2008.
- [5] P. Mahajan. Fully connected vs convolutional neural networks, 2020.
- [6] M. Sazli. A brief review of feed-forward neural networks. *Communications, Faculty Of Science, University of Ankara*, 50:11–17, 01 2006. doi: 10.1501/0003168
- [7] M. H. Winands. Monte-carlo tree search.
- [8] M. H. M. Winands. *Monte-Carlo Tree Search*, pp. 1–6. Springer International Publishing, Cham, 2015. doi: 10.1007/978-3-319-08234-9_12-1

A CONTRIBUTIONS

The logbook can be found here:

<https://docs.google.com/spreadsheets/d/1hpfvof2ZedxkvcbDp02qv2M49--vxD6UanxvZn39Jfe0/edit?usp=sharing>

B MEETING NOTES

List all the meeting notes. One note per subsection.

B.1 meeting notes November 12, 11h00

<https://drive.google.com/file/d/1hk85UKsrrY6vMWt10X9ohcbs3ZXE-b0S/view?usp=sharing>

B.2 meeting notes November 19, 10h00

https://drive.google.com/file/d/1eDX_Syj2Xkv6xpNSTwm_wlzNrK3M_zFB/view?usp=sharing

B.3 meeting notes November 26, 11h00

https://drive.google.com/file/d/1f06Tgugd-PVAAZ-x_mirV-8Q_zYbKfOL/view?usp=sharing

B.4 meeting notes December 3, 10h00

<https://drive.google.com/file/d/1LtCLWv6LEWxpCHMDGDUqCLQbv8hRQfmA/view?usp=sharing>

B.5 meeting notes December 10, 10h30

https://drive.google.com/file/d/15cGCJK_-JEEqbLGUj7U6nAsJX_3s4N0i/view?usp=sharing

B.6 meeting notes December 17, 10h00

https://drive.google.com/file/d/1JT06CbEwS3br_cKa1yIFKWXQEGi_nTcC/view?usp=sharing

B.7 meeting notes January 7, 10h00

https://drive.google.com/file/d/1jv_daNNrsGzvu3mLs_G6yVr-QFYKTTcN/view?usp=sharing

B.8 meeting notes January 14, 10h00

<https://drive.google.com/file/d/1UH3CcF4IBr0m52Y7eGUHIpExtfQkODwQ/view?usp=sharing>